Evaluating Dropout and Batch Normalization on LeNet-5

Final Project — Type-A, Topic 3

Course: Spring 2026 Introduction to Deep Learning
Name: Karim Smires
Due Date: May 13, 2026

Abstract

This project investigates the effect of two common regularization techniques—Dropout applied to fully-connected (FC) layers and Batch Normalization (BN) applied to convolutional layers—on a LeNet-5 network trained to classify handwritten digits from the MNIST dataset. Four configurations are evaluated in a 2×2 factorial design (Dropout on/off $\times$ BN on/off). Each configuration is trained with five independent random seeds to report statistically reliable mean and standard deviation of test accuracy. Results show that Dropout provides the larger accuracy gain ($\approx$0.18%), that BN provides a smaller but consistent benefit, and that combining both techniques yields the highest test accuracy of $99.14\% \pm 0.05\%$ while also achieving the lowest train-to-test generalization gap.

1 Introduction

Overfitting is a central challenge in training deep neural networks: a model may memorize the training set and fail to generalize to unseen data. Two widely adopted countermeasures are Dropout [2] and Batch Normalization [3]. Dropout randomly deactivates neurons during training, acting as an ensemble of many sub-networks. Batch Normalization normalizes the activations within each mini-batch, reducing internal covariate shift and providing a mild regularization effect.

While both techniques are well-studied individually, their interaction is less obvious: BN can reduce the need for Dropout in some settings, and Dropout can interfere with BN's running statistics if applied incorrectly. This project uses LeNet-5 on MNIST as a controlled testbed to answer the following question: *which combination of FC Dropout and convolutional BN produces the best generalization performance?*

2 Related Work

LeNet-5 [1] is a pioneering convolutional neural network introduced by LeCun et al. for handwritten digit recognition. It consists of two convolutional layers followed by three fully-connected layers and established the template for modern CNNs.

Dropout [2] was proposed by Srivastava et al. as a simple regularization method: during each training step, each neuron is independently zeroed with probability p, preventing co-adaptation of features.

Batch Normalization [3] was introduced by Ioffe and Szegedy to address internal covariate shift. By normalizing layer inputs over the mini-batch, it allows higher learning rates, reduces sensitivity to initialization, and acts as a regularizer. It has been shown to accelerate convergence, especially in convolutional networks.

3 Data Description

The MNIST dataset [1] consists of 70,000 grayscale images of handwritten digits (0–9), each of size 28×28 pixels.

- **Training set:** 60,000 images
- **Test set:** 10,000 images
- **Classes:** 10 (digits 0–9), balanced across splits

Images are normalized to zero mean and unit variance using the channel-wise statistics $\mu = 0.1307$, $\sigma = 0.3081$ prior to all experiments. No data augmentation is applied.

4 Method Description

4.1 Dropout

At each training iteration, every neuron in a targeted layer is independently set to zero with probability p (here $p = 0.5$). During inference the Dropout layer is disabled and activations are used as-is; PyTorch scales activations by $1/(1-p)$ during training so no additional rescaling is needed at test time. Dropout is applied after both hidden FC layers (FC1 and FC2) but not after the output layer.

4.2 Batch Normalization

For a mini-batch of activations $\{x_i\}_{i=1}^{N}$, BN computes:

$$\hat{x}_i = \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}}, \qquad y_i = \gamma \hat{x}_i + \beta$$

where $\mu_{\mathcal{B}}$ and $\sigma_{\mathcal{B}}^2$ are the mini-batch mean and variance, and γ, β are learned scale and shift parameters. BN is inserted after each convolutional layer and before the ReLU activation, following the convention of [3].

5 Model Description

The base architecture is a modern variant of LeNet-5 adapted for 28×28 MNIST input:

Layer	**Operation**	**Output Shape**	**Notes**
Input	—	$1 \times 28 \times 28$	
Conv1	Conv2d(1→6, 5×5)	$6 \times 24 \times 24$	[BN] → ReLU → MaxPool
Pool1	MaxPool2d(2×2)	$6 \times 12 \times 12$	
Conv2	Conv2d(6→16, 5×5)	$16 \times 8 \times 8$	[BN] → ReLU → MaxPool
Pool2	MaxPool2d(2×2)	$16 \times 4 \times 4$	
Flatten	—	256	
FC1	Linear(256→120)	120	ReLU → [Dropout]
FC2	Linear(120→84)	84	ReLU → [Dropout]
FC3	Linear(84→10)	10	Output logits

Table 1: LeNet-5 architecture. Bracketed components are conditionally present.

The four experimental configurations are:

1. **Config 1 — FC-Dropout + CONV-BN:** Dropout in FC1/FC2; BN in Conv1/Conv2.
2. **Config 2 — FC-Dropout + CONV-noBN:** Dropout in FC1/FC2; no BN.
3. **Config 3 — FC-noDropout + CONV-BN:** No Dropout; BN in Conv1/Conv2.
4. **Config 4 — FC-noDropout + CONV-noBN:** Baseline; no Dropout, no BN.

6 Experimental Procedure and Results

6.1 Experimental Setup

All experiments are implemented in PyTorch 2.11.0 and executed on an NVIDIA RTX 5070 GPU.

Hyperparameter	**Value**
Optimizer	Adam
Learning rate	0.001
Batch size	128
Epochs	20
Dropout rate	0.5
Random seeds	5 (seeds 0–4)

Table 2: Hyperparameters used across all configurations.

Each configuration is trained five times with different random seeds. Final test accuracy (epoch 20) and best test accuracy (highest over all epochs) are recorded per seed, and mean ± standard deviation are reported.

6.2 Results

Configuration	**Final Test Acc**	**Best Test Acc**	**Train Acc (ep.20)**	**Time**
Config 1: FC-Dropout + CONV-BN	$\mathbf{99.14\% \pm 0.05\%}$	$\mathbf{99.20\% \pm 0.03\%}$	98.94%	98.7 s
Config 2: FC-Dropout + CONV-noBN	$99.13\% \pm 0.05\%$	$99.19\% \pm 0.05\%$	98.89%	93.8 s
Config 3: FC-noDropout + CONV-BN	$98.96\% \pm 0.12\%$	$99.14\% \pm 0.06\%$	99.76%	101.3 s
Config 4: FC-noDropout + CONV-noBN	$98.94\% \pm 0.08\%$	$99.05\% \pm 0.04\%$	99.72%	103.1 s

Table 3: Test accuracy (mean ± std over 5 seeds) and final epoch training accuracy. Time is total wall-clock time for all 5 seeds.

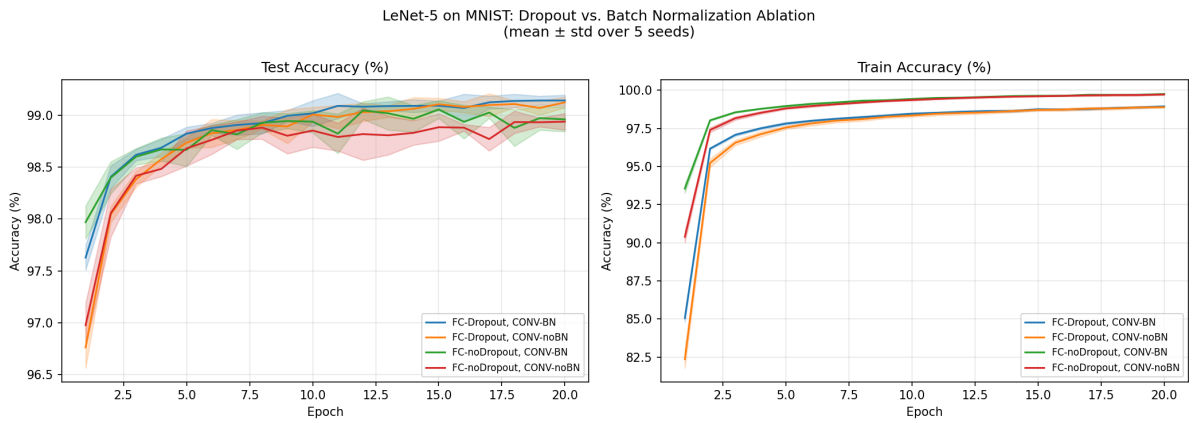

Figure 1: Test accuracy (left) and training accuracy (right) over 20 epochs. Shaded bands show ±1 standard deviation across 5 seeds.

6.3 Analysis

Config 1 (FC-Dropout + CONV-BN) achieves the highest performance in both final and best test accuracy, confirming that the combination of both regularization techniques is optimal.

Dropout has a larger impact than BN. Comparing configurations with and without Dropout (holding BN constant), Dropout contributes a ≈0.18% gain in test accuracy. The corresponding gain from BN alone (holding Dropout constant) is much smaller, at only ≈0.02%.

Dropout significantly reduces overfitting. Without Dropout (Configs 3 and 4), training accuracy reaches ≈99.7–99.8% while test accuracy plateaus near 98.9%, yielding a train-test gap of ≈0.8%. With Dropout (Configs 1 and 2), training accuracy converges near 98.9%, and test accuracy of ≈99.1–99.2% actually *exceeds* it—the expected behaviour of an effective regularizer.

BN provides a modest accuracy lift, but its stability effect is mixed. Holding the Dropout setting fixed, enabling BN improves the mean final test accuracy by about 0.02 percentage points in both comparisons. However, BN alone does not reduce variance: Config 3 (no Dropout, with BN) has a higher final-epoch standard deviation (0.12%) than Config 4 (0.08%). The strongest overall result is still the combined setting in Config 1, which pairs the best mean accuracy with low variance.

7 Conclusion

This study evaluated four configurations of Dropout and Batch Normalization on LeNet-5 trained on MNIST. The results clearly demonstrate that:

1. **Config 1 (FC-Dropout + CONV-BN) is the best-performing configuration**, achieving 99.14% ± 0.05% final test accuracy and 99.20% ± 0.03% peak test accuracy.
2. Dropout is the dominant regularizer in this setting, contributing a larger accuracy improvement than BN alone.
3. BN provides only a small secondary accuracy benefit in this setup, and its effect on stability is mixed rather than uniformly positive across configurations.
4. Omitting both techniques (Config 4) leads to the worst generalization, confirming that regularization is necessary even on a simple dataset like MNIST.

These findings align with established literature and provide empirical confirmation that, for this LeNet-5/MNIST experiment, combining Dropout and Batch Normalization gives the strongest overall performance.

References

[1] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.

[2] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: A simple way to prevent neural networks from overfitting," *Journal of Machine Learning Research*, vol. 15, pp. 1929–1958, 2014.

[3] S. Ioffe and C. Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift," in *Proc. International Conference on Machine Learning (ICML)*, pp. 448–456, 2015.

A Source Code

```python
import torch.nn as nn


class LeNet5(nn.Module):
    """
    LeNet-5 with configurable Batch Normalization (conv layers) and Dropout (FC layers).

    Configurations tested:
        1. use_dropout=True,  use_bn=True   -> FC-Dropout + CONV-BN
        2. use_dropout=True,  use_bn=False  -> FC-Dropout + CONV-noBN
        3. use_dropout=False, use_bn=True   -> FC-noDropout + CONV-BN
        4. use_dropout=False, use_bn=False  -> FC-noDropout + CONV-noBN

    Architecture (MNIST 28x28 input):
        CONV1: 1->6,  5x5  -> [BN] -> ReLU -> MaxPool2d(2)  => 6x12x12
        CONV2: 6->16, 5x5  -> [BN] -> ReLU -> MaxPool2d(2)  => 16x4x4
        Flatten: 256
        FC1: 256->120 -> ReLU -> [Dropout]
        FC2: 120->84  -> ReLU -> [Dropout]
        FC3: 84->10
    """

    def __init__(self, use_dropout: bool = True, use_bn: bool = True, dropout_rate: float = 0.5):
        super().__init__()
        self.use_dropout = use_dropout
        self.use_bn = use_bn

        def conv_block(in_ch, out_ch):
            layers = [nn.Conv2d(in_ch, out_ch, kernel_size=5)]
            if use_bn:
                layers.append(nn.BatchNorm2d(out_ch))
            layers += [nn.ReLU(), nn.MaxPool2d(kernel_size=2)]
            return layers
```

```python

        self.conv = nn.Sequential(
            *conv_block(1, 6),
            *conv_block(6, 16),
        )

        fc_layers = [
            nn.Linear(256, 120), nn.ReLU(),
        ]
        if use_dropout:
            fc_layers.append(nn.Dropout(dropout_rate))
        fc_layers += [nn.Linear(120, 84), nn.ReLU()]
        if use_dropout:
            fc_layers.append(nn.Dropout(dropout_rate))
        fc_layers.append(nn.Linear(84, 10))

        self.fc = nn.Sequential(*fc_layers)

    def forward(self, x):
        x = self.conv(x)
        x = x.view(x.size(0), -1)
        return self.fc(x)
```

Listing 1: model.py — LeNet-5 definition

```python
"""
Final Project --- Type-A Topic 3
Evaluating Dropout (FC layers) and Batch Normalization (Conv layers) on LeNet-5 / MNIST

Runs 4 configurations across NUM_SEEDS random seeds, reports mean +/- std accuracy,
and saves training curves to results.png and raw numbers to results.json.
"""

import json
import random
import time

import matplotlib.pyplot as plt
import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader
from torchvision import datasets, transforms

from model import LeNet5

# -- Hyperparameters ------------------------------------------------------------
LEARNING_RATE = 0.001
BATCH_SIZE    = 128
EPOCHS        = 20
DROPOUT_RATE  = 0.5
NUM_SEEDS     = 5  # seeds 0-4; increase for tighter error bars

CONFIGS = [
    {"use_dropout": True,  "use_bn": True,  "label": "FC-Dropout, CONV-BN"},
    {"use_dropout": True,  "use_bn": False, "label": "FC-Dropout, CONV-noBN"},
    {"use_dropout": False, "use_bn": True,  "label": "FC-noDropout, CONV-BN"},
    {"use_dropout": False, "use_bn": False, "label": "FC-noDropout, CONV-noBN"},
]
```

```python

COLORS = ["#1f77b4", "#ff7f0e", "#2ca02c", "#d62728"]

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")


# -- Reproducibility ------------------------------------------------------------
def set_seed(seed: int):
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    torch.cuda.manual_seed_all(seed)
    torch.backends.cudnn.deterministic = True


# -- Data ------------------------------------------------------------------------
def get_loaders():
    transform = transforms.Compose([
        transforms.ToTensor(),
        transforms.Normalize((0.1307,), (0.3081,)),
    ])
    train_loader = DataLoader(
        datasets.MNIST(root="./data", train=True,  download=True, transform=transform),
        batch_size=BATCH_SIZE, shuffle=True,  num_workers=4, pin_memory=True,
    )
    test_loader = DataLoader(
        datasets.MNIST(root="./data", train=False, download=True, transform=transform),
        batch_size=BATCH_SIZE, shuffle=False, num_workers=4, pin_memory=True,
    )
    return train_loader, test_loader


# -- Training / evaluation -------------------------------------------------------
def train_epoch(model, loader, optimizer, criterion):
    model.train()
    correct, total, total_loss = 0, 0, 0.0
    for images, labels in loader:
        images, labels = images.to(device), labels.to(device)
        optimizer.zero_grad()
        out  = model(images)
        loss = criterion(out, labels)
        loss.backward()
        optimizer.step()
        total_loss += loss.item() * labels.size(0)
        correct    += (out.argmax(1) == labels).sum().item()
        total      += labels.size(0)
    return 100.0 * correct / total, total_loss / total


def eval_epoch(model, loader, criterion):
    model.eval()
    correct, total, total_loss = 0, 0, 0.0
    with torch.no_grad():
        for images, labels in loader:
            images, labels = images.to(device), labels.to(device)
            out  = model(images)
            loss = criterion(out, labels)
            total_loss += loss.item() * labels.size(0)
```

```
            correct    += (out.argmax(1) == labels).sum().item()
            total      += labels.size(0)
    return 100.0 * correct / total, total_loss / total


# -- Single config run across all seeds ----------------------------------------
def run_config(cfg, train_loader, test_loader):
    """Returns train_accs and test_accs arrays of shape (NUM_SEEDS, EPOCHS)."""
    train_accs = np.zeros((NUM_SEEDS, EPOCHS))
    test_accs  = np.zeros((NUM_SEEDS, EPOCHS))

    for s in range(NUM_SEEDS):
        set_seed(s)
        model     = LeNet5(use_dropout=cfg["use_dropout"], use_bn=cfg["use_bn"],
                           dropout_rate=DROPOUT_RATE).to(device)
        criterion = nn.CrossEntropyLoss()
        optimizer = optim.Adam(model.parameters(), lr=LEARNING_RATE)

        for epoch in range(EPOCHS):
            tr_acc, tr_loss = train_epoch(model, train_loader, optimizer, criterion)
            te_acc, te_loss = eval_epoch(model, test_loader, criterion)
            train_accs[s, epoch] = tr_acc
            test_accs[s,  epoch] = te_acc
            print(f"  [{cfg['label']}] seed={s} epoch={epoch+1:2d} | "
                  f"train={tr_acc:.2f}%  test={te_acc:.2f}%")

    return train_accs, test_accs


# -- Plotting ------------------------------------------------------------------
def plot_results(all_results):
    epochs = np.arange(1, EPOCHS + 1)
    fig, axes = plt.subplots(1, 2, figsize=(14, 5))
    fig.suptitle("LeNet-5 on MNIST: Dropout vs. Batch Normalization Ablation\n"
                 f"(mean +/- std over {NUM_SEEDS} seeds)", fontsize=12)

    for i, (cfg, (train_accs, test_accs)) in enumerate(zip(CONFIGS, all_results)):
        c = COLORS[i]
        for ax, accs, split in zip(axes, [test_accs, train_accs], ["Test", "Train"]):
            mean, std = accs.mean(0), accs.std(0)
            ax.plot(epochs, mean, color=c, label=cfg["label"])
            ax.fill_between(epochs, mean - std, mean + std, alpha=0.2, color=c)

    for ax, title in zip(axes, ["Test Accuracy (%)", "Train Accuracy (%)"]):
        ax.set_xlabel("Epoch")
        ax.set_ylabel("Accuracy (%)")
        ax.set_title(title)
        ax.legend(fontsize=8, loc="lower right")
        ax.grid(True, alpha=0.3)

    plt.tight_layout()
    plt.savefig("results.png", dpi=150)
    print("\nSaved -> results.png")


# -- Main ----------------------------------------------------------------------
```

```python
def main():
    print(f"Device : {device}")
    print(f"Epochs : {EPOCHS}  |  Batch: {BATCH_SIZE}  |  LR: {LEARNING_RATE}")
    print(f"Seeds  : {NUM_SEEDS}  |  Dropout rate: {DROPOUT_RATE}\n")

    train_loader, test_loader = get_loaders()

    all_results = []
    summary     = []
    total_start = time.time()

    for cfg in CONFIGS:
        print(f"\n{'='*60}")
        print(f"  {cfg['label']}")
        print(f"{'='*60}")
        t0 = time.time()
        train_accs, test_accs = run_config(cfg, train_loader, test_loader)
        elapsed = time.time() - t0

        final_mean = test_accs[:, -1].mean()
        final_std  = test_accs[:, -1].std()
        best_mean  = test_accs.max(axis=1).mean()
        best_std   = test_accs.max(axis=1).std()

        all_results.append((train_accs, test_accs))
        summary.append({
            "label":          cfg["label"],
            "final_acc_mean": round(float(final_mean), 4),
            "final_acc_std":  round(float(final_std),  4),
            "best_acc_mean":  round(float(best_mean),  4),
            "best_acc_std":   round(float(best_std),   4),
            "elapsed_s":      round(elapsed, 1),
        })
        print(f"\n  Final test acc: {final_mean:.2f}% +/- {final_std:.2f}%  "
              f"| Best: {best_mean:.2f}% +/- {best_std:.2f}%  | {elapsed:.1f}s")

    total_elapsed = time.time() - total_start

    # -- Summary table ------------------------------------------------------
    print(f"\n{'='*70}")
    print(f"{'SUMMARY':^70}")
    print(f"{'='*70}")
    print(f"{'Configuration':<35} {'Final Acc':>12} {'Best Acc':>12} {'Time':>8}")
    print(f"{'-'*70}")
    for s in summary:
        print(f"{s['label']:<35} "
              f"{s['final_acc_mean']:.2f}%+/-{s['final_acc_std']:.2f}%  "
              f"{s['best_acc_mean']:.2f}%+/-{s['best_acc_std']:.2f}%  "
              f"{s['elapsed_s']:>6.1f}s")
    print(f"\nTotal experiment time: {total_elapsed:.1f}s")

    # -- Save raw results ---------------------------------------------------
    results_json = {
        cfg["label"]: {
            "train_accs": train_accs.tolist(),
            "test_accs":  test_accs.tolist(),
        }
        for cfg, (train_accs, test_accs) in zip(CONFIGS, all_results)
    }
```

```python
    results_json["summary"] = summary
    with open("results.json", "w") as f:
        json.dump(results_json, f, indent=2)
    print("Saved -> results.json")

    plot_results(all_results)


if __name__ == "__main__":
    main()
```

Listing 2: main.py — Experiment runner